

I agree that the method cannot be too complex, and is most likely to be almost as simple as k3. For example, k3 was double transposition with different parameters, so k4 is likely double or triple something else with different parameters.

Here is an example of something that I think is too complex:

10 digit binary necklace with 5 1s and 5 0s each. There are exactly 26 :

```
{"0000011111", "0000101111", "0000110111", "0000111011", "0000111101",  
"0001001111", "0001010111", "0001011011", "0001011101", "0001100111",  
"0001101011", "0001101101", "0001110011", "0001110101", "0001111001",  
"0010010111", "0010011011", "0010011101", "0010100111", "0010101011",  
"0010101101", "0010110011", "0010110101", "0011001101", "0011010101",  
"0101010101"}
```

as the kryptos alphabet in the same 5x5 grid.

```
KRYPT  
OSABC  
DEFGH  
IJLMN  
QUVWX  
Z
```

and using a key stream, ks, that could be a repeated password, or a continually increment string, or some already encrypted text on the sculpture, or a memorized quote.

first write the pt on each line (row headers) then write out the binary for each letter to the right, then write the ks to the right, then write the row number r and column number c of the ks to the right, then write the binary number with the rth 0 and the cth 1 flipped (for 6,0 dont flip any bits), then write the binary again starting with the first largest group of zeros with the smallest group of zeros to its circular left, then finally write out the binary for that letter .

for example,

```
t 0000111101 k 11 1000011101 0000111011 p  
h 0001111001 e 32 0011011001 0010011011 j  
e 0001101101 y 13 1001101001 0011001101 w  
k 0000011111 z 60 0000011111 0000011111 k
```

This would not be equivalent to poly-alphabetic substitution with 26 random looking alphabets, if for example, we add a conditional step at the end that for each pt position with the condition that if both the last two corresponding positions of the ks are each either vowels or any of the letters in kryptos, then the bits are flipped.

We could also not do any bit flipping at all and just transpose the bits, there would be 10! ways of doing this but we could just transpose the two bits identified in the key stream rather than flip them. However, since the output is invariably not in the smallest shifted position as it is in the table, no matter what we do, we need to calculate the decryption manually for each possible ct letter and ks or position so that is 100 inverses which are nearly all unique. All ct letters at their position decipher to exactly the pt letters at their position. But it takes 10 calculations and comparisons to determine the inverse for each of these 100 ct ks letter pairs.

The k in the pt would be unchanged in the ct for a z in the ks. At all possible steps, there are exactly 5 1s and 5 0s, all pt gives a single ct, and the decryption is a different 5x5 box that is the reverse of the transpose of the encryption one but nonetheless outputs the pt exactly as it was, so things are consistent.

The reason why I think this example is too complex because for each letter enciphered there is way too many characters that need to be written in intermediate steps, in this case, more than 30 at least. So it would be $30 \times 100 + 26 + 26 \times 10$ or about 3,500 = 35 times the total number of k4 characters of writing to encipher for k4. It would take about two to three minutes on average per character or about two hours for 100 characters.

However, a variant of the enigma machine with three rotors and no reflector that can be done very practically easily and quickly with paper and pencil is possible and is certainly not complex in this respect.

Without a reflector it is simply just substitution in succession with three different rotating alphabets, each of which are rotating at different, but constant rates. The three alphabets can be three different single strips of graph paper with 26 squares each that is the alphabet copied twice as well as one for the alphabet. For example, the kryptos alphabet, the kryptos alphabet reversed, and the alphabet reversed, and the alphabet itself.

Now we just need the rotor stepping and the starting vector. This would be a two 3 character strings. Note that each step is one letter, however we could also add another three characters for a different number of characters stepped by. We could also have it more simple like mod 2 or mod 3 etc.

First write the pt with each character on a new line, next write the starting vector next to the first pt character. next copy each character in the starting vector on the first line the corresponding number of times in the corresponding character in the rotor stepping, then increment, and repeating.

Now to encipher any pt character, first put each of the three alphabet strips with the first letter of each strip under the corresponding letter of the normal alphabet strip. Now find the pt character in the normal alphabet and find the letter exactly below it in the first alphabet strip, now repeat this process with the new letter, find it in the normal alphabet and return the letter below it in the second alphabet strip, now repeat with this newer letter, find it in the normal alphabet and return the letter below it in the third alphabet strip.

For example, if the stepping is abc and the starting is kry

kry
lry
msy
nsz
otz
...

ABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZ
-----KRYPTOSABCDEFGHIJLMNQUVWXZ
-----ZXWVUQNMLJIHGFEDCBASOTPYRK
-----ZYXWVUTSRQPONMLKJIHGFEDCBA

Deciphering is just the strips in the same position only with the steps of letter following in reverse starting with the third strip to normal then second strip to normal then first strip to normal which is the deciphered letter, very fast and simple indeed.

I don't think this is complicated because there is not much that needs to be written. So it would be $5 \cdot 100 + 26 \cdot 4$ or about 600 = 6 times the total number of k4 characters of writing to encipher for k4.

Now considering that it is not even necessary to write ANY of the three letters, (since alphabets are fixed unless they need to be shifted and the shifts can be indicated on the pt with just slashes or accents between letters where a alphabet needs to be shifted and the alphabets to be shifted indicated by small circles at the top middle and bottom of the slash respectively, and this is extremely fast and simple) at all it turns out to be about $2 \cdot 100 + 26 \cdot 4$ or about 300 = 3 times the total:

So enigma with 3 rotors and no reflector is particularly elegant short and simple while still being very hard to crack, and I think is a very very strong possibility and not at all too complicated for a non mathematician since there are a small number of very simple steps, for each square move the strip by 1 if indicated by the corresponding mark on the pt, then just follow that letter with a finger through the steps and write down the ct letter.

Nothing additional needs to be written down for the decryption.

Essentially the method just transposes the alphabets. The method is logical and sound.

It would take about 20 seconds per character or about 30 minutes for 100 characters.

combination of transposition and substitution is still possible.

Just break up k4 into blocks and transpose and encrypt the blocks.

For example if blocks are of length 4:

block 17 is characters 65-68 YPVT ERLI
block 18 is characters 69-72 TMZF NCLO

or length 3
or length 5
...

Or just use blocks of length 1 and substitute the letter for 3 numbers, ($3^3=26+1$),
then transpose the numbers,
then substitute the new numbers back into the pt letter
